MTP F26 · TEAM C · RADIO TEAM → PROTOCOLS & SOFTWARE TEAM

Protocol and firmware proposal

From Radio Team (Santiago, Andrian, Marcos) **To** Ibrahim, Guillem, Sofia

Version 0.4 · Sun 18 Oct 2026

New in v0.4 (18 Oct): the boxes run on a Raspberry Pi 3 with Linux instead of the Pico 2 (the team's request; analysis on the Team C Pi 3 Switch page). The protocol, frames, NM spec and all C modules stay. What changes on your side:

- A Linux **HAL**: spidev with one ioctl per radio command, CE and IRQ through libgpiod, busy-wait microsecond delays.
- A Linux **environment**: timerfd + epoll, and a radio thread with real-time priority on an isolated core.
- Files on the **SD card** instead of raw flash, with fsync; read-only root so a power cut cannot corrupt the card.
- The stick on the Pi's own USB port: our FatFs code reads the raw block device, so the 109 checks stay valid. TinyUSB and PIO-USB are gone.
- A Linux boot takes 10–30 s, so a reboot costs more. New simulator test **T19**: MRM with a 30 s reboot still delivers the file in 90.8 s. Target a boot of 10 s or less.
- Goodput may drop from 107 to about **97 kbps** if the gap between frames grows to 0.5 ms on Linux. That is still well above what we need.

New in v0.3.7 (17 Oct): a Protocol specs section that says where every interface is defined. Among them is the **NM interop spec** to share with the other teams: radio registers, frame, gossip rules and a test vector. The NM channel moves to **RF_CH 77**, off the BLE advertising channel at 2480 MHz. Fixes from the 16-Oct code review:

- a **POLL** frame recovers a lost ACK without resending the whole burst;
- a box completes the NM-file only when the whole-file CRC matches;
- the codec now streams through 4 KB buffers;
- `usb_msc` handles sticks formatted on a Mac or a PC (GPT, UTF-8 names; 109 checks);
- the throughput model now counts the header and the gap between frames: **~107 kbps** goodput, not 159;
- the simulator models the hop clocks on both boxes.

New in v0.3.6 (16 Oct): Network Mode now has link-protocol diagrams and a **relay log**: each box writes NM_LOG.TXT next to its copy of the NM-file, as evidence that it took part. The `usb_msc` module is started: stick I/O over FatFs (FAT and exFAT), 96 host checks plus a cross-check against a second FAT implementation; the Pico glue is a skeleton. MRM also has an **option B**: 20 dBm on one fixed channel without hopping, only if Atenea Q11 says yes.

New in v0.3.5 (16 Oct): every team uses 20 dBm, so every team hops in MRM; no fixed channel plan. Our hop set grew to 40 channels (1.0 s cycle, the worst-case re-sync time). Random hop collisions with the other teams can cost about a third of our ACKs; new simulator test T14 (30 % loss both ways) still delivers all lines in 81 s.

New in v0.3.4 (16 Oct): a battery check for the `ui` and `store` modules: read the cell on ADC3, warn below 3.5 V, warn loudly below 3.4 V at setup (never block a round), and below 3.2 V write received data to flash after every burst so a brown-out loses nothing.

New in v0.3.3 (16 Oct): SRI and MRM hop. A professor approved 20 dBm e.i.r.p., which our radio may use only while **hopping** every 25 ms over 40 channels and **listening before talking** on each hop (EN 300 328 adaptive FHSS). The protocol stays the same; the radio layer hops. Two things are now **required** on your side: a **slot clock** shared by both ends (each frame carries the slot number) and **hop acquisition** (the receiver parks on one channel until it hears the sender). The driver already provides `nrf24_cca()` and `nrf24_fhss_channel(slot)`, and the simulator runs it with `--fhss`. NM does not hop, and Quick Mode can stay on one fixed channel at 10 mW.

New in v0.3: all four teams moved to 2.4 GHz. The radio is now the nRF24L01+ (Ebyte E01-ML01SP4, one per box, one omni antenna; decided 13 Oct) at 250 kbps, so **frames are at most 32 B**, each team has its own channel in MRM, and there is no duty-cycle limit any more. The protocol code and the simulator were ported; all 11 scenarios pass.

The radio team has fixed the physical layer: 2.4 GHz, nRF24L01+, 250 kbps, with a tested driver. Everything above it belongs to protocols & software. This is a starting **proposal** built from the competition rules and our link numbers, to speed you up. It is not a decision. Please change whatever you see better.

In short

1. **Compress the file as one deflate stream.** On English text in 100-byte lines it measured **~3×**. The whole file then crosses in **~25 s instead of ~75 s** (model at 107 kbps, no loss), which leaves most of the 2 minutes to recover from attacks. The simulator, at 5 % loss, needs 27 s with compression and 71 s without.
2. **Selective-repeat ARQ** with bursts of 16 frames and one bitmap ACK. Frames are delivered strictly in order, because only the **first unbroken run of correct lines** counts.
3. **Resume after any interruption.** A HELLO handshake identifies the file, and the receiver keeps its progress, so an "attack" or a reboot costs seconds, not the round.
4. **Network Mode: gossip, not routing.** Every node rebroadcasts the small file (~20 frames) on a randomised timer (Trickle-style). No addresses or positions are needed, and every node ends up with a copy (that is how teams win NM). It needs agreement with all teams.
5. **Specs:** every interface has one file that defines it (Protocol specs). The NM interop spec is the one to share with the other teams.
6. **Software: our C code as one Linux program on the Raspberry Pi 3**, started by systemd at boot. The radio loop runs in a real-time thread on its own core; USB, files and compression run in the main thread. The **PC simulator** runs the same protocol code, so it is tested before boards exist.

Protocol specs: where each one lives

Each interface has one source of truth. The pages explain it; the files below define it. **Only the NM spec goes to the other teams**. The rest is internal to Team C, although the repo is public.

SPEC	WHERE (SOURCE OF TRUTH)	COVERS	AUDIENCE	STATUS
NM interop spec v1.0	NM Interop Spec page · `radio/NM_SPEC.md`	Radio registers (hex), NM frame, gossip rules, CRC-16, test vector, relay log, interop test	**All teams**: share this one	draft, to agree by 23
SRI/MRM link frames	`proto/src/frames.h` · `link.h`	DATA, ACK, HELLO, HELLO_ACK, END, POLL; timers; ARQ and resume (frames, procedure)	Team C (both boxes)	in the simulator

SPEC	WHERE (SOURCE OF TRUTH)	COVERS	AUDIENCE	STATUS
Radio profiles	radio/phy_config.py → radio/fw/nrf24_config.h	Register values per mode (TEAM_250K, TEAM_FHSS, NM_COMMON, option B), hop order, COT and idle times, power caps	Team C; NM_COMMON is copied into the NM spec	checked, 318 driver t
Radio driver API	radio/fw/nrf24.h	`nrf24_send`, `nrf24_poll_rx`, `nrf24_cca`, `nrf24_fhss_channel`, the HAL to port	PT + RT	host-tested
Codec	proto/src/codec.h	One deflate stream through 4 KB buffers, file to file (system zlib)	Team C	in the simulator
USB stick I/O	proto/src/usb_msc.h	Input-file choice, load, safe save (.part → verify → rename), FAT/exFAT, MBR/GPT sticks	Team C	host-tested; Linux bl
Relay log format	proto/src/nm_log.h	NM_LOG.TXT written next to the NM-file	Team C (others may copy it)	in the simulator

Environment contract: `proto/src/proto_env.h` (on the Pi: timerfd + epoll for timers, `getrandom` for random numbers). Call `on_tx_idle` after every transmission, and `send` must queue a frame, never drop it. Test commands: `make -C proto test` and `make -C radio/fw test`.

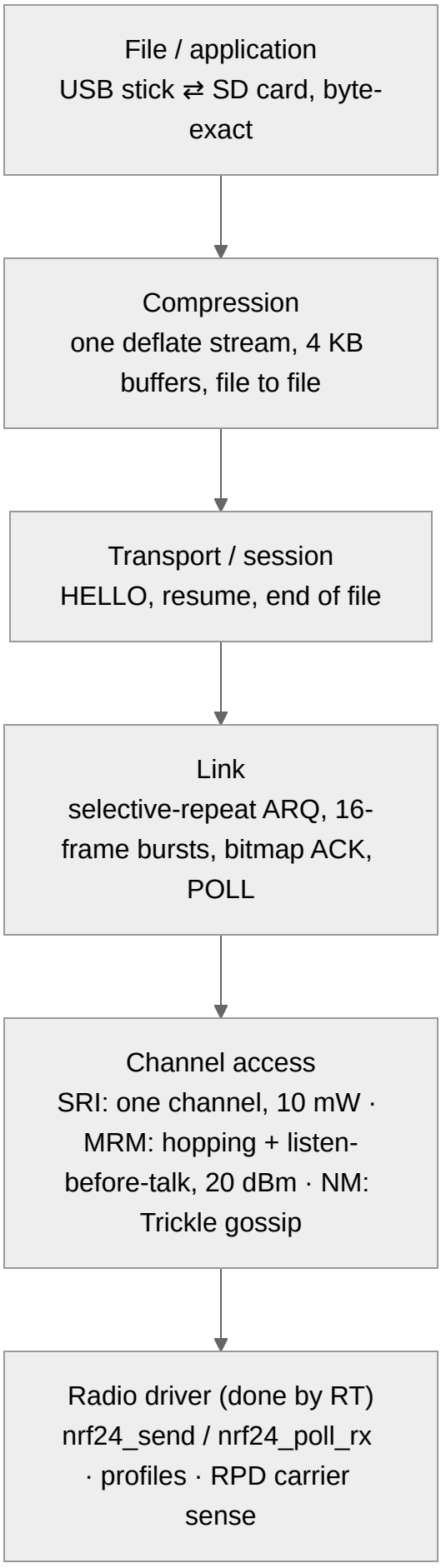
What the rules ask of the software

RULE (COMPETITION PDF)	CONSEQUENCE FOR THE PROTOCOL
File: UNICODE .txt, ~1 MB, ~10 000 lines of equal length, "return"-separated; may contain redundancy	Transfer the file **byte-exact**: no re-encoding, keep the BOM and line endings. "UNICODE" may mean UTF-16, which compresses even better (4.7× measured).

RULE (COMPETITION PDF)	CONSEQUENCE FOR THE PROTOCOL
Only identical lines count, and **only the first continuous range** of correct lines	Strict in-order delivery. Never skip a lost frame to send newer data, and stop when the file is complete.
5 min setup while wrapped in foil; the professor then picks the TX device at random; GO/STOP by button; 3 min to upload	Load the file into **both** devices at setup. A role switch (TX/RX) is read when the button is pressed. Save the received file to USB after STOP.
"The system should be designed to self-recover from eventual interruptions on the link"; "attacks" during the round	Retry forever with backoff, resume by HELLO, watchdog. Received data persists across a reboot.
NM: random order of teams on two rims, 5 lines / 0.5 KB, 5 min; win = deliver a copy from one of your nodes	A flooding/gossip protocol shared by all teams, where every node stores the file. **No switch may indicate the position in the network.**
MRM: all teams transmit at once, 260 m, interference	Channel-access layer (listen before talk, backoff) and a plan agreed with the other teams. SRI and MRM hop (D-R6): the driver forces a clear-channel check before each hop's first frame and skips busy hops.
Quick Mode: one line (< 500 chars), zero errors, < 2 min, first attempt only	A simple, robust first version is enough. Reliability beats speed there.
CNAF UN-85 (radio team): SRI/MRM hop at ≤ 20 dBm e.i.r.p. (a), NM at 10 mW on one channel (b); no duty-cycle limit; nRF24 frames ≤ 32 B	No transmit-time budget. Headers must stay small: every byte of header is 3 % of a frame. Hops last 30 ms: one 16-frame burst plus its ACK fits one hop.

Protocol stack

The radio team delivers the bottom layer, ready to use. Protocols & software owns the rest.


File / application
USB stick ⇄ SD card, byte-exact
Compression
one deflate stream, 4 KB buffers, file to file
Transport / session
HELLO, resume, end of file
Link
selective-repeat ARQ, 16-frame bursts, bitmap ACK, POLL
Channel access
SRI: one channel, 10 mW · MRM: hopping + listen-before-talk, 20 dBm · NM: Trickle gossip
Radio driver (done by RT)
nrf24_send / nrf24_poll_rx · profiles · RPD carrier sense

Frame formats

Each radio frame carries at most **32 bytes** (nRF24 dynamic payload). A full DATA frame takes **1.32 ms** on air at 250 kbps.

type|flags 1 B	session 1 B	seq 2 B	compressed file bytes ≤ 28 B	CRC 2 B · radio

DATA, SRI/MRM. 32 B at most; the radio adds and checks the CRC (profile TEAM_250K). The 16-bit seq covers files up to 1.83 MB. *session* is a random byte chosen at HELLO, so frames from earlier runs or other teams are dropped.

type|flags 1 B	session 1 B	next_seq 2 B	bitmap 2 B	link 1 B	CRC 2 B · radio

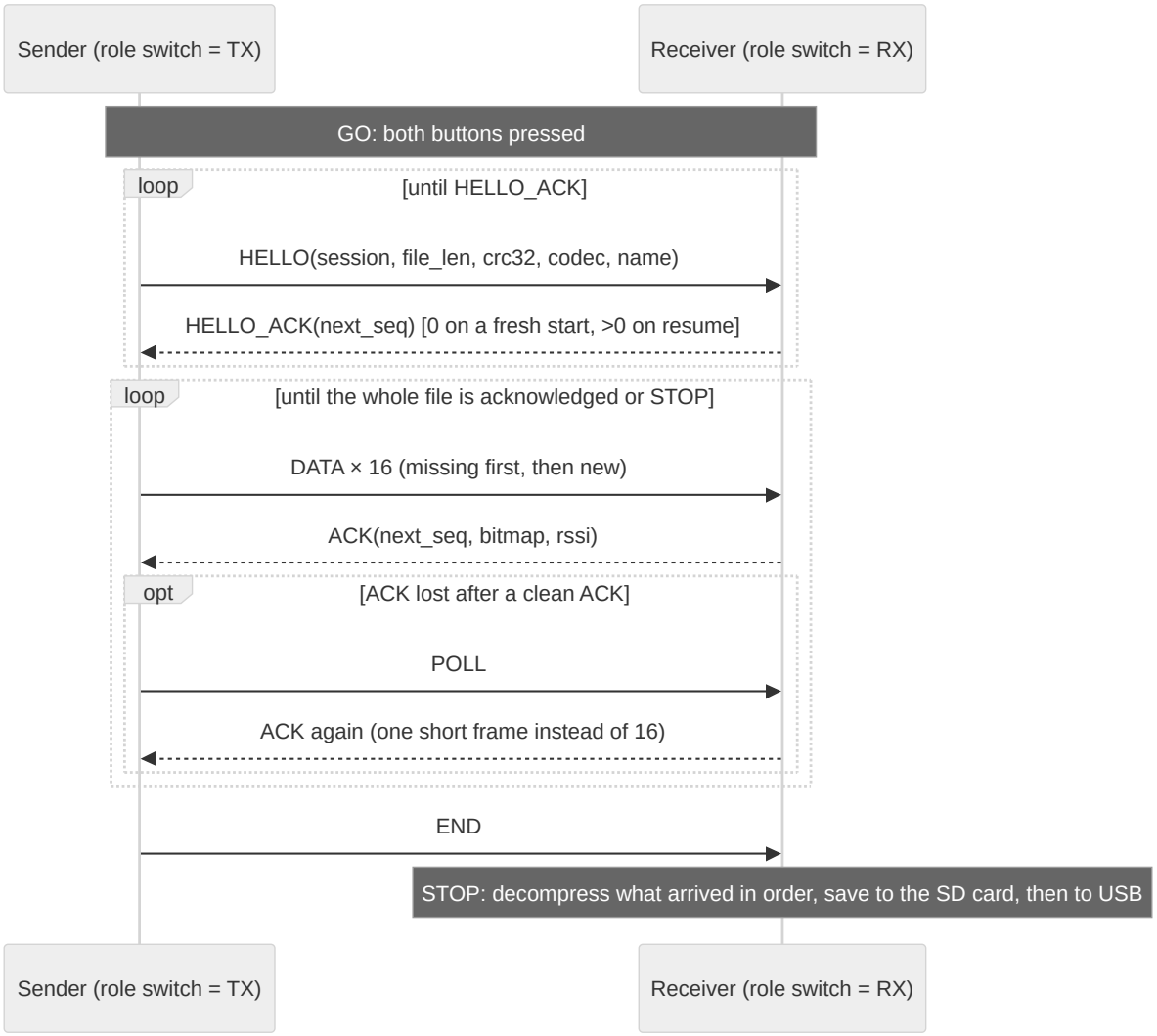
ACK. next_seq = every frame before it has arrived. bitmap bit i = frame next_seq+1+i arrived. The last byte is for link logging: the nRF24 has no RSSI, so it carries the RPD bit (signal above about −76 dBm at the antenna).

type 1 B	session 1 B	file_len 4 B	file_crc32 4 B	codec 1 B	name ≤ 21 B	CRC 2 B · radio

HELLO (sender) and HELLO_ACK (receiver; replaces name with next_seq). They identify the file, so a rebooted receiver can resume where it stopped.

Type values (proposal): 0x01 DATA, 0x02 ACK, 0x03 HELLO, 0x04 HELLO_ACK, 0x05 END, 0x06 POLL (2 B: type, session; "repeat your ACK"), 0x4E NM_DATA. Flags: last-in-burst, retransmission. Definitions: `proto/src/frames.h`.

Link procedure (SRI and MRM)



Timing

- Burst of 16 × 32 B frames: **≈ 26 ms** (1.32 ms each + 0.3 ms gap)
- ACK + 2 radio turnarounds: **≈ 2.5 ms**
- ACK timeout: **≈ 11.6 ms** at 250 kbps (scales with bit rate), then resend the burst
- Goodput (model: 28 B of file per frame, 0.3 ms gap): **≈ 107 kbps** on one channel, **≈ 96 kbps** hopping with 10 % of hops busy

Resend rules

Each burst first resends every frame the last bitmap marked missing, then fills with new frames. The window never advances past a gap, so the receiver's output is always one continuous run.

If the last ACK showed a clean window, an ACK timeout most likely lost only the ACK: send a 2-byte **POLL** first, and resend the burst only if that times out too.

After 8 silent timeouts, send HELLO again with backoff. 250 kbps is already the nRF24's

slowest rate, so there is no slower fallback. Never give up before STOP.

Recovering from attacks

Jamming or shadowing only looks like lost ACKs, and the retry loop covers it. A reboot of either box is handled by HELLO. The receiver keeps `next_seq` and the data on the SD card (fsync before it counts as committed), and the sender re-reads the file from the card.

Hardware watchdog through systemd (`RuntimeWatchdogSec`). A Linux reboot takes 10–30 s, not 0.3 s: simulator T19 (MRM, 30 s down) still delivers the file in 90.8 s. Target a boot of 10 s or less.

Compression: measured

Measured on 7 Oct with Python's zlib, lzma and bz2. The corpus was the English text of the course rules and slides, laid out as equal 100-byte lines (55 KB, no repetition). The real file is unknown and "may feature redundancy", so treat these as a floor. Re-run on any sample file we get.

METHOD	UTF-8	UTF-16LE	NOTES
deflate, one stream (zlib level 6–9)	2.97×	4.7–4.8×	Recommended. The receiver inflates progressively as frames arrive in order.
deflate, independent 32 KB blocks	2.74×	4.31×	Not needed: after a reboot the receiver keeps its received prefix on the SD card and resumes the same stream.
deflate, independent 4 KB blocks	2.23×	3.32×	Only needed if delivery were out of order. It isn't.
deflate, independent 1 KB blocks	1.89×	2.56×	

METHOD	UTF-8	UTF-16LE	NOTES
lzma / bz2, whole file	3.3×	6.3–6.4×	Better ratio. Too heavy for the Pico, but feasible on the Pi 3 (1 GB RAM): about 10 % fewer bytes in UTF-8 and about 25 % fewer in UTF-16 than deflate. Worth trying once the deflate path works; the receiver can still decode the prefix it got.

> On the Pi 3, compress during the 5-minute setup with the system zlib. `codec_deflate_stream` still streams file to file through 4 KB buffers; with 1 GB of RAM that is no longer necessary, but it does no harm. Deflating 1 MB at level 9 takes under a second on the Pi's Cortex-A53. The receiver inflates its received prefix from the SD card straight to the USB stick at STOP.
> **Raw fallback** (`codec = NONE`): only if deflate fails. The 16-bit sequence number covers 1.83 MB, so a raw 2 MB UTF-16 file would deliver only its first 1.83 MB (simulator T16: 8 448 of 10 000 lines).

EFFECT PER 2-MIN ROUND (1 MB FILE, 250 KBPS, 107 KBPS GOODPUT)	TIME TO SEND ALL OF IT (MODEL, NO LO
No compression	≈ 75
4 KB independent blocks (2.2×)	≈ 34
deflate stream (3.0×; 2.60× on the repo sample)	≈ 25 s (29 s at 2.6
deflate stream at 1 Mbps (SRI option)	≈ 11

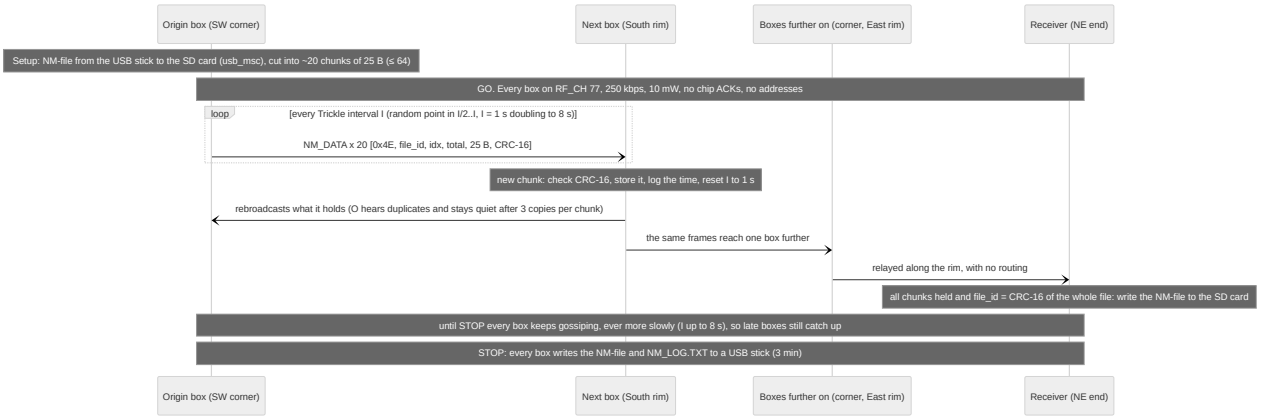
Network Mode: gossip proposal for all teams

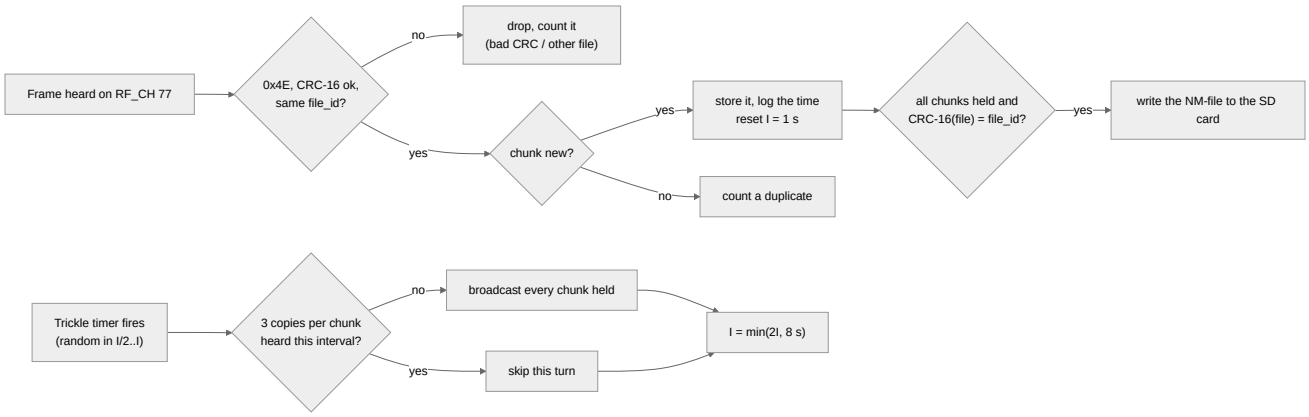
No node knows its position, and the rules forbid a position switch. The file is only about 500 B, or 20 frames. So we suggest that nothing is routed: **every node that holds part of the file rebroadcasts it** on a randomised timer, following the Trickle algorithm (IETF RFC 6206), until STOP.

type 0x4E 1 B	file_id 2 B	idx 1 B	total 1 B	file bytes ≤ 25 B	CRC-16 sw 2 B

NM_DATA: **the inter-team standard** on the NM_COMMON radio profile. file_id = CRC-16 of the whole file. The radio's CRC protects each hop; the software CRC (CCITT-FALSE, big-endian) checks the data end to end across relays.

Link protocol: how the NM-file moves





Behaviour

- The sender (SW corner) starts with the whole file.
- Every node keeps every valid chunk with the same `file_id`. When it hears its first new chunk, it starts rebroadcasting everything it holds.
- Rebroadcast interval: random in [I/2, I], starting at I = 1 s and doubling up to 8 s. Skip a turn if 3 or more identical copies were heard in that interval.
- Once all `total` chunks are held and the CRC-16 of the whole file equals `file_id`, the node writes the file to the SD card;

Why it fits

- **No addresses or positions**, so it works in any random order of teams.
- **Every node ends up with a copy**, which is the win condition. That makes the +2 bonus, and the +0.5 "poker bonus" for all teams, likely.
- TX time: about 33 ms per full rebroadcast; in the simulator the busiest node transmits about 1.1 s in 5 min. No duty-cycle limit at 2.4 GHz.
- Simple enough for every team to implement on any nRF24L01+ board.

otherwise it starts over. It saves the file to USB after STOP.

To agree with the other teams

- Everything is in the NM interop spec (`radio/NM_SPEC.md`): frame, radio registers, timers, and a test vector each team can check its code against.
- Interop test by Fri 6 Nov: every team's box in one room on the NM channel (RF_CH 77, 2477 MHz); full rehearsal Fri 20 Nov.
- Protocols & software leads this with the other teams' protocol people; the radio team handles the radio settings.

Proof of participation: the relay log

The rules make a team an NM winner by *"delivering a copy of the NM-file from at least one of their node devices"*, and only identical files count. So the copy is the proof, and gossip gives every box a full copy. As supporting evidence, each box also writes `NM_LOG.TXT` next to it: which file it holds (file_id, CRC-32), when its chunks first arrived, and what it rebroadcast. Built by `proto/src/nm_log.c` from the gossip state; the simulator writes one per node to `build/nm_logs/`. Example, a relay box in the harsh simulator run:

```
MTP-F26 NM relay log
team: Team C   box: C2   firmware: v0.1
role: relay
NM-file: COMPLETE, id 0xCCE6, 20 chunks, 500 bytes, CRC-32 8C4AFEBA
copy on this stick: NM_FILE.TXT
times from GO: first frame heard 2.874 s, file complete 36.099 s, last frame heard 296.
relayed: 48 broadcasts, 853 frames sent, first 3.461 s, last 295.316 s
heard: 20 new chunks, 1278 duplicates, 0 bad CRC, 0 other file; 0 broadcasts suppressed
chunk first held (s from GO):
 0:11.865 1:5.732 2:36.099 3:10.427 4:7.702 5:9.123 6:5.735 7:27.294
 8:7.705 9:13.479 10:10.435 11:28.072 12:2.874 13:28.075 14:5.539 15:13.486
 16:10.441 17:13.489 18:11.888 19:5.742
```

Still open: the output file name and encoding (Atenea Q9). Write the NM-file to the SD card (with fsync) the moment it completes, not only at STOP, so a brown-out cannot lose it.

MRM: every team hops

All teams transmit at the same time, 260 m apart. Since 16 Oct every team uses 20 dBm, which narrowband radios may only do while hopping, so **every team hops** with listen-before-talk (D-R6). Ours uses 37 channels (the even RF_CH 2–80, minus the three BLE advertising channels) in its own order (stride 7). There is no fixed channel plan any more.

- The risk is a neighbouring team's box a few metres away: at 10 m it arrives 41 dB above our partner (47 dB at a two-ray peak), so hops landing within 4 MHz of each other collide (about 13 % of hop pairs). Pessimistic cost: ~31 % lost ACKs at the sender end, ~2 % lost data at the receiver end; even far apart, ~7 % of ACKs meet another sender on the same channel. Hence the POLL. Keep ≥ 10 m between teams' boxes, 30 m between senders (radio brief, MRM section).
- **Listen before talk** with the RPD bit is now mandatory in SRI/MRM: the driver refuses to transmit on a hop without a clear check, and a busy hop is skipped.
- **Option B** (only if Atenea Q11 says yes): the same 20 dBm on **one fixed channel** without hopping (profile `TEAM_20DBM_FIXED`, MRM plan 24 / 49 / 74 / 82). No hop clock or acquisition needed; the driver still demands a clear-channel check before every ≤ 10 ms of transmission (about 6 frames), so a 16-frame burst needs 3 checks. EN 300 328 limits non-hopping radios to 10 mW/MHz, which a narrow signal exceeds, hence the question.

Software ask: build the slot clock and hop acquisition early (bench by week 8), and keep the **fixed-channel mode** (Quick Mode, fallback) selectable at run time.

Software architecture (Raspberry Pi 3, Linux)

MODULE	RUNS ON	JOB	OWNER
`radio/fw/nrf24 + crc16`	radio thread	Radio driver, profiles, RPD carrier sense, clear-channel check and hop order for SRI/MRM hopping. **Done, 318 host checks; Linux HAL started 18 Oct.**	RT
`board_hal`	radio thread	**Started 18 Oct** (`radio/fw/linux/`, builds with the strict flags, not yet run on a Pi; bench tool `nrf24_probe`). `nrf24_hal_t` on Linux: spidev with a buffer transfer (one ioctl per command; CSN = CE0 driven by the kernel), CE and IRQ through libgpiod with kernel timestamps (hop acquisition needs them), `delay_us` as a busy-wait on `CLOCK_MONOTONIC` (usleep overshoots)	RT + PT

MODULE	RUNS ON	JOB	OWNER
`link`	radio thread	Frames, bursts, ACK bitmap, POLL, timeouts (`proto/src/link.c`, in the simulator)	PT
`session`	radio thread	HELLO / resume / END, role from the switch	PT
`codec`	main thread	zlib deflate at setup, streamed file to file; inflate on the receiver at STOP (`codec.c`)	PT
`store`	I/O thread	Files on a small ext4 data partition for the file sent and the file received, with a progress record: `pwrite` + `fdatasync` off the radio thread (an SD sync takes 10–100 ms), `committed` advanced only after the sync; below 3.2 V sync after every burst	PT
`usb_msc`	main thread	**Started 16 Oct:** portable part done (`proto/src/usb_msc.c`, FatFs R0.15a, FAT and exFAT, MBR and GPT sticks as macOS and Windows format them, UTF-8 file names; 109 host checks, cross-checked with pyfatfs). We bring our own FAT32/exFAT sticks. On the Pi, keep FatFs and give it the stick as a raw block device (`/dev/sda`, no automount, O_DIRECT reads, disk group): a small `usbm_disk_t`, and the tested code stays as it is. The Pico glue (TinyUSB + PIO-USB) is obsolete	PT
`nm_gossip`	radio thread	Trickle timers, chunk store, relay trace for NM_LOG.TXT (`nm_log.c`)	PT
`ui`	main thread	Button, role and mode switches, LED patterns through libgpiod (the rules require a blinking pilot light); battery check from the MAX17048 fuel gauge on I2C (the Pi has no ADC), thresholds re-measured at T0 (≥ 3.5 V ok, 3.4–3.5 V warning pattern, < 3.4 V at setup loud warning, never block a round)	PT

> Linux pitfall: the scheduler. A stock kernel can pause our process for up to about 1.1 ms; the receiver's idle-ACK timer (~4 ms) and the TX_DS timeout (~3.8 ms) are tight against that. Use a PREEMPT_RT kernel, run the radio thread SCHED_FIFO on an isolated core (`isolcpus=3`, `mlockall`), set `core_freq=250` so the SPI clock stays fixed, or widen both timers on Linux. Keep the root file system read-only (overlayfs) so a power cut or the drop test cannot corrupt the SD card. Test this early, because it is the classic way to lose a round.

Simulator: results on the nRF24 radio updated 17 Oct

The radio team built the simulator in `proto/`. It runs the **same C protocol code** that will run on the Pi 3 (link with resume, NM gossip, deflate) against a modelled channel:

- half-duplex radios, with airtime from the real driver; in hopping mode, each box has its own slot clock with crystal drift, and the receiver parks and re-syncs;
- collisions, and random or bursty loss;
- a jamming blackout, and reboots.

Scoring copies the judges: decompress, then count the first unbroken run of identical lines. `make -C proto test` runs 21 scenarios plus 109 USB checks in about 2 seconds.

SCENARIO (1 MB SAMPLE, DEFLATE 2.62×, 250 KBPS, SEED 1)	LINES	WHO
Clean channel	10 000	
5 % random loss	10 000	
Bursty fades (~6 % loss)	10 000	
10 s jamming attack + 5 % loss	10 000	37 s · back 0.2 s after th
Receiver or sender reboot at 15 s, 5 % loss (0.3 s down)	10 000	
Same, but 30 s down (a Linux boot)	10 000	
1 Mbps (SRI option), 5 % loss	10 000	
No compression, 5 % loss	10 000	
NM, 10 nodes, 10 % / 70 % loss at 1 / 2 hops	all nodes	
NM, 10 nodes, 30 % / 95 % loss	all nodes	
Hopping + listen-before-talk (MRM baseline), 5 % loss, 10 % of hops busy	10 000	
Hopping, 25 % of hops busy, receiver reboot at 15 s (re-sync)	10 000	
MRM, all four teams hopping at 20 dBm: 5 % data loss, 30 % ACK loss, 10 % of hops busy	10 000	

SCENARIO (1 MB SAMPLE, DEFLATE 2.62×, 250 KBPS, SEED 1)	LINES	WHO
T19: MRM as above, receiver (Pi 3) reboots at 15 s and is down 30 s	10 000	
Stress: hopping with 30 % loss both ways	10 000	115 s (just insid
10 seeds each: 5 % loss / hopping with 10 % busy	10 000	worst 27.5 s
NM spec test vector (the frames in the NM spec) decoded by our gossip code	–	

> **It already caught a bug** (on the old 868 MHz radio). With fixed timeouts, the 38.4 kbps fallback delivered only 883 lines. One frame (15.2 ms) outlasted the receiver's 15 ms idle-ACK timer, so the receiver ACKed in the middle of bursts and went deaf. All timeouts are now derived from the bit rate.

Still open in the simulator: an MRM scenario with four teams' boxes modelled explicitly (today their effect is a loss rate), and loss taken from measured packet-error rates instead of fixed percentages. Assumptions to measure on the Pi: 1 ms turnaround, 0.3 ms gap between frames (0.5 ms would give ~97 kbps; 1 ms gap and 2 ms turnaround ~73 kbps, still above the 67 kbps an uncompressed file needs). Reboot downtime is set with `--reboot-down`. The hop clock and acquisition exist **only in the simulator** so far; they still have to be built on the Pi.

Milestones

BY	PROTOCOLS & SOFTWARE	WITH
Fri 16 Oct	Agree or amend this proposal; set up a shared git repo; simulator skeleton	RT
Fri 23 Oct	Link + session in the simulator; agree the NM interop spec with the other teams	RT (NM radio settings)
Fri 30 Oct	On the bench Pi: Linux HAL, USB stick → SD card; one line over the air	ET, RT
Fri 6 Nov	**Quick Mode attempt** (one error-free line, first try); NM interop test with the other teams	all
Fri 20 Nov	Compression, full SRI throughput, resume after reboot, NM rehearsal; go/no-go for MRM hopping	RT, other teams

BY	PROTOCOLS & SOFTWARE	WITH
Fri 27 Nov	MRM channel access; field tests at 70 m and 260 m	RT
Fri 4 Dec	Pretest, fully boxed	all

What we need from protocols & software

1. Agree or counter-propose the stack, frames and compression by **Fri 16 Oct**.
2. An owner for the NM talks with the other teams. Send them the NM interop spec once the page is shared.
3. The Linux base, together with electronics: Pi OS Lite (64-bit) with a PREEMPT_RT kernel (our suggestion), or Buildroot if the boot must get under 5 s.
4. Two questions for Marcos to add to the Atenea list: is a TX/RX **role switch** allowed (it does not encode network position)? What file name should the received file have on the USB stick?

Related pages: NM Interop Spec v1.0 (for all teams) · Team C Radio Brief v0.11 (radio settings, hopping, test plan) · Team C Hardware Proposal v0.5 (Pi 3 + E01-ML01SP4 HAT) · Team C Pi 3 Switch. Rules quoted from *MTP F26 – Program Competition Rules – Network Upgrade III + Sustainability*.

References: RFC 6206, The Trickle Algorithm · zlib · Linux spidev · libgpiod · Pi real-time kernel latency · FatFs.